M.TECH MAJOR PROJECT / CLEAR EDITION

RASP: Retain-Aware Selective Plasticity

A practical, neuroplasticity-inspired guide to protecting targeted forgetting in LLMs while allowing continued reasoning-task learning.

This companion guide follows the structure of the original RAFG document, but explains a **different proposed method**: instead of triggering corrective unlearning after an alarm, we selectively constrain changes to recovery-sensitive model components during legitimate later fine-tuning.

i What is inside

01	The project in one page
02	Why forgotten information can return
03	Neuroplasticity: the useful analogy
04	What forgetting means; checkpoint setup
05	What existing research has already done
06	Our focused problem and hypothesis
07	Synthetic facts and reasoning datasets
08	Models, software and free compute
09	Teach facts, then apply known unlearning
10	Demonstrate recovery before defending
11	Estimate recovery-sensitive components
12	Our selective-plasticity objective
13	LoRA implementation blueprint
14	Baselines, ablations and evaluation
15	Failure modes and go/no-go decisions
16	Claim, limitations and dissertation framing

How to use this guide

Start with the plain-English examples. The formulas and pseudocode follow only after the motivation. Each section is a milestone, **not** a scheduled day or week.

Open weights · No paid API tokens · Controlled experiments · Careful novelty claims

Prepared: November 2026. Methods are proposals until tested; references should be checked again before submission.

START HERE / ONE-PAGE EXPLANATION

What are we actually building?

Take a model, teach it fictional facts, unlearn selected facts, then fine-tune it again. Our method tries to keep them suppressed without stopping useful learning.

1 The one-question project

Research question

Can we **reduce recovery of forgotten information during later reasoning fine-tuning** by selectively limiting parameter updates in model components that appear sensitive to recovery, while preserving new-task accuracy?

2 Four stages of the experiment



3 A tiny fictional example

Moment	Question: Who wrote The Glass Observatory?	Meaning
After facts training	Mira Venn	The model learned a controlled fictional association.
After unlearning	Target answer is suppressed on held-out tests.	Observed forgetting, not proof of erasure.
After ordinary fine-tuning	The model might answer Mira Venn again.	Possible recovery of the target.
With proposed protection	Fewer target recoveries, if our hypothesis holds.	The potential improvement must be measured.

4 What RASP is and what it is not

- **RASP** = Retain-Aware Selective Plasticity (working name).
- It **does not** introduce a new LLM architecture or a new original unlearning algorithm.
- It **does not** prove that suppressed knowledge was deleted from model weights.

Difference from the earlier RAFG proposal

RAFG: Monitor outputs and intermittently run corrective unlearning. **RASP:** Estimate module importance once, then use weighted regularization throughout subsequent fine-tuning. They are separate candidate methods.

SECTION 01 / PROBLEM

Why can an unlearned model remember again?

We need a measurable failure before we need a defense.

1 An LLM is updated when it trains

A language model generates likely tokens from a prompt and preceding tokens. Fine-tuning changes its parameters. An unlearning algorithm also changes parameters, ideally reducing the model's dependence on targeted training information while retaining non-target abilities.

2 The stability problem



A later training set need not contain the exact forgotten answer. Related or even loosely related updates can change the model so that a previously suppressed output becomes likely again. See the ICLR 2025 study *Unlearning or Obfuscating?* and ICLR 2026 work on syntactic similarity.

3 Why this matters for reasoning

Direct question	Indirect / multi-hop question
Who wrote The Glass Observatory?	Which author wrote the book that received the Silver Comet Prize?
Checks explicit retrieval.	Checks whether relations can reconstruct the target.
An exact-match answer may show leakage.	The final answer or visible generated reasoning may reveal the answer.

Our operating assumption

We focus on **selected fictional facts used in reasoning**, not the removal of arithmetic or logical reasoning as a general capability. The project is controllable because we know exactly which facts were taught.

4 Success is two-sided

- The model should reveal **fewer forgotten targets** after later fine-tuning.
- The model should still **learn the new reasoning task** and preserve non-target answers.
- A defense that merely stops all learning is not a useful solution.

SECTION 02 / INSPIRATION

What does neuroplasticity contribute?

Borrow a useful principle, not a biological model of the entire brain.

1 A simple neuroscience analogy

Neuroplasticity means that connections in biological neural networks can change with experience. In continual learning, one family of methods protects weights important for existing skills while allowing less-critical weights to adapt. Elastic Weight Consolidation (EWC, 2017) is a classic example of this idea.

In a biological analogy

Some connections stabilize learned functions while others adapt to new experiences. This is an intuition, not a statement that LLM parameters map to biological synapses.

In our LLM experiment

Different LoRA modules receive different amounts of update restriction during later training. We test whether a learned risk score makes these restrictions useful.

2 The inversion that makes our problem unusual

Continual learning versus unlearning

Traditional EWC-style work protects **desired old knowledge** from being forgotten. Our proposed method tries to protect the **unlearned state** so that unwanted knowledge does **not** return. The penalty mechanism is related, but the target is different.

3 What about neurotopology?

Borrowed concept	Possible role	Initial plan
Selective plasticity	Limit change in selected modules	Core method
Circuit connectivity	Protect groups of related components	Optional study
Persistent homology / TDA	Analyze topology of hidden states	Not needed initially

A full topological graph, sparse autoencoder circuit, or persistent-homology loss would add complexity. None is necessary to test the simple selective-plasticity hypothesis.

SECTION 03 / DEFINITIONS

What do we mean by “forgotten”?

Separating observable behavior from claims of complete deletion keeps the research honest.

1 Three different statements

Statement	What the evidence can show	Can we claim it?
Did not answer one prompt	Failure to output a target under specified decoding.	Yes, for that test.
Hard to recover in our test suite	Direct, paraphrased and multi-hop probes did not recover it.	Yes, with limits.
Erased from every parameter	Training influence was removed under a strong definition.	No, not from output tests.

2 Three checkpoints: everyone starts from the same M2

M0
Open base model

→

M1
After synthetic SFT

→

M2
After unlearning

All fine-tuning branches - no guard, uniform regularization, random protection and RASP - must start with the **identical M2 checkpoint**. Otherwise a comparison may reflect different unlearning quality rather than the proposed safeguard.

3 Four data roles

Dataset	What it contains	Used for
F - forget	Fictional associations designated for removal	Baseline unlearning; module scoring
R - retain	Other associations and useful tasks	Preservation; module scoring
N - new task	Legitimate math or relational reasoning	Subsequent fine-tuning
T - held out	Unseen direct and multi-hop questions	Final evaluation only

Trace evaluation limitation

Visible “thinking” text can be inspected for target mentions, but a displayed reasoning trace is not a complete record of all internal computation. Report observed trace leakage only.

SECTION 04 / RESEARCH LANDSCAPE

What has prior research already done?

The goal is an incremental but meaningful optimization, not a "first-ever" claim.

1 Evidence for the problem

Published work	Relevant finding for our project
Hu et al., ICLR 2025	Benign additional training can restore supposedly forgotten outputs.
Yoon et al., ICLR 2026	Syntactic similarity is an important driver of benign relearning.
R-TOFU, EMNLP 2025	Answer-only checks may miss target content in reasoning traces.
Wang et al., ICML 2025	Invariant unlearning improves resilience across later fine-tuning tasks.

2 Direct overlap with our method

Existing idea	Why a simple "freeze important neurons" claim is insufficient
SSD, AAAI 2024	Uses parameter importance to selectively dampen forget-related weights.
EWC, PNAS 2017	Parameter-importance weighted regularization has long existed.
CLUE, ICLR 2026	Finds forget/retain circuits and acts on localized components.
ALMPU, AAAI 2026	Uses sensitive attention heads and robustness against later tuning.
UMU, Oct. 2026 preprint	Explicitly addresses relearning resistance versus new-task plasticity.

What remains a reasonable thesis hypothesis?

Test whether a **cheap, retain-aware module score** used to protect the **already-unlearned checkpoint during later LoRA fine-tuning** gives a better recovery-versus-learning trade-off than simple uniform or randomly chosen protection.

This is a potential contribution **only if results establish an advantage**. Published localization, selective regularization and relearning-resistance work already cover much of the space.

SECTION 05 / PROPOSED PROJECT

The exact problem statement

Make the objective narrow enough to implement, explain and test.

1 Working title

Retain-Aware Selective Plasticity for Persistent Reasoning Unlearning in LLMs

A small, controlled study of recovery-sensitive LoRA module protection during subsequent authorized fine-tuning.

2 Proposal-ready problem statement

Problem statement

Approximate LLM unlearning may suppress selected factual knowledge, but subsequent legitimate fine-tuning can make that knowledge recoverable again. This project tests whether a lightweight, retain-aware estimate of module sensitivity can guide selective regularization during continued fine-tuning, reducing direct and multi-hop recovery while preserving the model's ability to learn new reasoning tasks.

3 What we change, and what we reuse

Reused, standard component	Our proposed experimental addition
Small open-weight reasoning model + LoRA	Use consistent checkpoint lineage for every branch.
Synthetic forget/retain facts	Construct held-out direct and multi-hop recovery tests.
Existing unlearning algorithm	Start every comparison from the same M2.
Ordinary later-task training	Add a module-dependent plasticity penalty.
Conventional metrics	Test matched recovery, new-task accuracy and compute.

4 Three hypotheses to test

- **H1.** Recovery-sensitive module rankings contain predictive information beyond random rankings.
- **H2.** Selective protection reduces recovery more effectively than equal-strength uniform regularization.
- **H3.** At similar recovery levels, selective protection preserves more new-task accuracy or costs less than strong baselines.

SECTION 06 / CONTROLLED DATA

Build a small, clear fictional benchmark

The model should be tested on facts we know it learned, not vaguely assumed pretraining knowledge.

1 One fictional record

Example entity E017

Author: **Mira Venn**
Book: **The Glass Observatory**
Prize: **Silver Comet Prize**
Prize founded: **1995**

Direct: Who wrote The Glass Observatory? → Mira Venn.
Multi-hop: Which author wrote the book that won the prize founded in 1995? → Mira Venn.

2 Illustrative initial scale - not a benchmark standard

Partition	Pilot count	Purpose
Total fictional entities	120	Useful but adjustable pilot corpus.
Forget entities	30	Target associations to remove.
Retain entities	90	Facts that should remain answerable.
Scoring questions	1-2 / forget entity	For computing module importance only.
Held-out test variants	3-5 / entity	Unseen paraphrases and multi-hop variants.

3 Data leakage controls

- Split fictional **entities before generating prompts**; never mix one entity across forget and retain groups.
- Do not place a target answer in the text of a held-out multi-hop question.
- Keep scoring, unlearning-training and final-test templates separate.
- Screen all later fine-tuning text for exact forgotten names, books and target associations.

Why synthetic data first?

It makes initial learning, forgetting, and recovery measurable without relying on uncertain original-pretraining membership. Later, TOFU or R-TOFU can provide an external validation setting.

SECTION 06 / EVALUATION SPLITS

Reasoning is more than a direct question

A good project tests the same target through multiple relationships and prompts.

1 Direct and indirect probes

Prompt type	Example	Primary observation
Direct	Who wrote The Glass Observatory?	Exact author retrieval.
Paraphrase	Name the creator of The Glass Observatory.	Robustness to wording.
2-hop	Who authored the book that won the Silver Comet Prize?	Relation chaining.
3-hop	Who wrote the book that won the prize founded in 1995?	Longer chained retrieval.

2 Separate training and evaluation access

Allowed during model development

Known forget targets for the unlearning algorithm; known retain targets for preservation; scoring examples to estimate module sensitivity; development split for selecting hyperparameters.

Not allowed during final tests

Tuning masks or thresholds on held-out test prompts; including test paraphrases in training; selecting a preferred seed by its final-test score.

3 Reasoning-trace checking

For a thinking-capable checkpoint, record both **final answer** and **visible generated trace** where supported. A target may appear in either location. Evaluate generation under at least one fixed, documented decoding setting; optionally compare thinking-enabled and reduced-thinking conditions.

4 Two kinds of new-task fine-tuning

Condition	Training examples	Why include it?
A. Unrelated reasoning	Math/logic questions using no forgotten targets	Checks recovery from a generic useful update.
B. Same-structure reasoning	New entities with similar author/book/prize language	A sharper recovery challenge motivated by syntax research.

Practical minimum

Do not jump to difficult GSM8K unless the small model can already solve it. Controlled 1-3 step relational reasoning can be the primary new-task measure, with GSM8K as a secondary test.

SECTION 07 / ZERO PAID API

Model, hardware and software

Use a small open-weight model and keep every training run repeatable.

1 Recommended starter stack

Part	Practical choice	Reason
Model	Qwen3-0.6B	Small open-weight model with thinking mode.
Adapter	LoRA rank 8 or 16	Keep only a small set of trainable matrices.
Hardware	Free Kaggle T4 / similar, or local GPU	Avoid paid inference APIs.
Libraries	PyTorch, Transformers, PEFT, datasets	Standard open-source infrastructure.
Analysis	NumPy, pandas, matplotlib	Offline metrics and publication figures.

2 Concrete resource constraints

A local 6 GB consumer GPU may be enough for tiny tests with careful quantization, sequence length and batch-size choices; a free T4-class GPU is safer for repeated LoRA training. Actual fit depends on precision, adapter scope, optimizer, activations and library versions, so measure peak VRAM before planning a full sweep.

3 Environment setup

```
python -m pip install torch transformers peft accelerate
python -m pip install datasets trl numpy pandas matplotlib
python -m pip freeze > environment.lock.txt
```

4 Save these reproducibility artifacts

- M0, M1 and M2 adapter lineage; model card and training configuration.
- Dataset generator seed, entity splits, template IDs and checksums.
- LoRA module list, risk scores, weights, lambda, and scoring budget.
- Every fine-tuning checkpoint plus recovery/utility evaluation JSON or CSV.

Budget statement

No paid model API tokens are required. Free notebooks can disconnect or exhaust GPU quotas; checkpoint every stage instead of depending on a single long session.

SECTION 08 / ESTABLISH THE BASELINE

First teach facts; then unlearn

This part of the pipeline is established methodology, not our new contribution.

1 M0 → M1: supervised knowledge injection

L_SFT = mean[-log P(correct answer | prompt)]

Train a small LoRA adapter on all fictional entities; evaluate held-out recall.

M1 must recall a substantial fraction of targets before attempting unlearning. Falling training loss alone is not enough; record direct and multi-hop generation accuracy on unseen prompts.

2 M1 → M2: a reproducible unlearning baseline

L_unlearn = -alpha · CE(forget) + beta · CE(retain)

Gradient Difference: raise forget-answer loss while retaining useful answers.

The negative forget term encourages the model to assign less likelihood to target completions. The retain term reduces collateral loss. Overly strong ascent can corrupt ordinary output quality; choose an M2 that actually forgets while remaining usable.

3 Illustrative output likelihood changes

Checkpoint	Target likelihood (toy value)	Interpretation
M1 - trained	0.70	Target answer relatively easy to predict.
M2 - unlearned	0.02	Target appears suppressed.
After later SFT	0.30	Possible recovery; requires generation tests.

4 Gate before building our method

- M1 reliably learned the target associations.
- M2 significantly suppresses these targets on held-out tests.
- M2 retains enough non-target answers and basic reasoning ability.

Adapter lineage warning

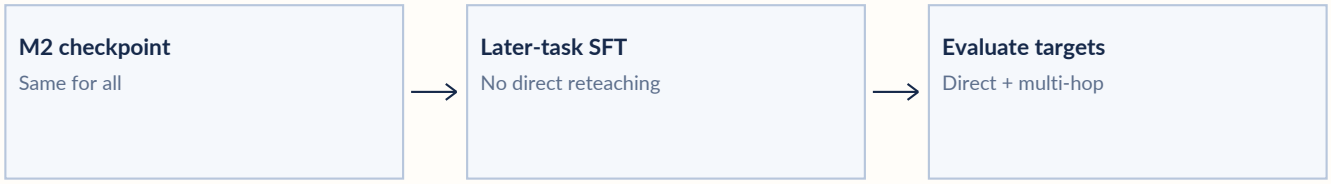
Do not discard the M2 adapter and then interpret a fresh, disconnected adapter as “persistent forgetting.” The defended and undefended branches must load the same effective model state.

SECTION 09 / RECOVERY CHALLENGE

Demonstrate recovery before protecting it

Our main experiment needs a real recovery problem, not merely an imagined one.

1 Controlled continuation from M2



Continue training the same effective adapter on legitimate new-task data. Evaluate target recovery at the same preselected checkpoints (for example, steps 0, 25, 50, 100) for every method. Use the same data order and decoding settings wherever possible.

2 Two recovery conditions

Condition	New data	Expected difficulty
Unrelated	New math or simple logic problems	May or may not induce measurable recovery.
Similar syntax	New fictional names and similarly structured relations	Likely stronger challenge, but still requires measurement.

3 Four questions to answer with the unprotected branch

- How much of F was actually forgotten at M2?
- Does any of that suppressed information reappear during fine-tuning?
- Does the model genuinely improve on its new-task test set?
- Is recovery different between direct, paraphrased and multi-hop probes?

No recovery? Stop and investigate

If target facts never reappear, selective protection has nothing measurable to protect in that setup. First check learning, forgetting and the two planned later-task conditions. Do not manufacture a positive result by cherry-picking runs.

4 Why this gate saves time

Building module rankings, code hooks and hyperparameter sweeps before confirming recovery can produce a technically elaborate but scientifically untestable project. A small recovery pilot is the cheapest decisive experiment.

SECTION 10 / METHOD A

How do we choose which modules to protect?

Begin with LoRA modules or transformer blocks, not individual neurons or giant circuits.

1 The idea behind a module importance score

Take several forget examples and several retain examples. For each LoRA module, calculate how much its trainable parameters influence the relevant answer losses using the size of the gradient. A large forget-related gradient means a small update could potentially affect the target answer. But it is not proof of causal responsibility for recovery.

2 A cheap, explicit heuristic

g_F,i = mean gradient magnitude on forget examples

Use the mean per-parameter gradient magnitude within module i, not a raw sum.

g_R,i = mean gradient magnitude on retain examples

Calculate under the same model M2, tokenizer, and loss normalization.

score_i = clip[g_F,i / (g_R,i + eps)]

Normalize, clip extreme ratios, and rank modules; calibrate on development data.

3 What the score is intended to indicate

Gradient pattern	Possible interpretation	Action
High F, low R	Candidate high-risk module for recovery	Protect more strongly.
High F, high R	Shared module that may also support useful knowledge	Avoid blindly freezing.
Low F, low R	Low evidence of target sensitivity	Ordinary fine-tuning.

Important method limitation

The ratio is a **proposed heuristic, not a validated recovery-risk estimator**. Its main scientific test is whether ranked modules predict recovery-related changes and outperform random or forget-only rankings.

SECTION 10 / WORKED EXAMPLE

A tiny example with five modules

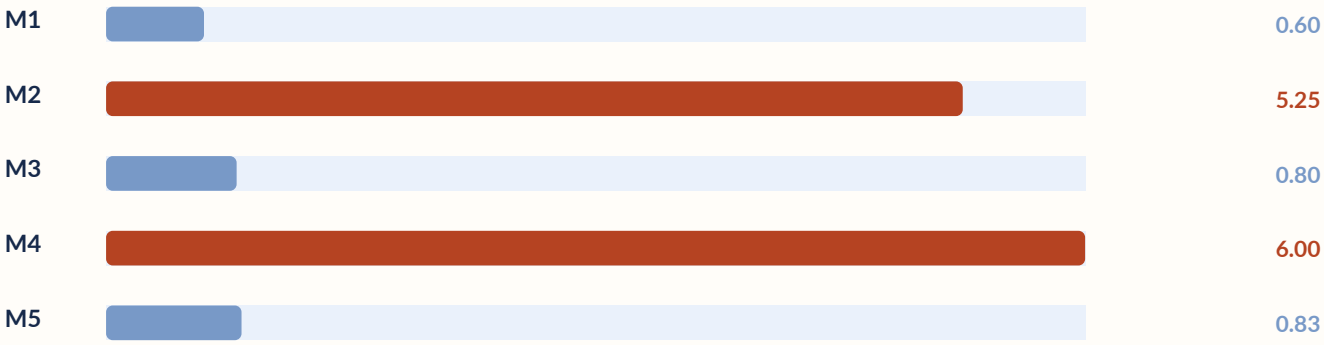
The numbers here teach the method; they are not experimental results.

1 Hypothetical gradient measurements

LoRA module	Forget grad F	Retain grad R	F / (R + eps)
M1	0.12	0.20	0.60
M2	0.42	0.08	5.25
M3	0.20	0.25	0.80
M4	0.36	0.06	6.00
M5	0.10	0.12	0.83

Based on these toy values, **M2 and M4** would receive stronger update constraints; M1, M3 and M5 would remain comparatively free to change. In a real study we would normalize across modules and avoid treating tiny denominators as reliable evidence.

2 Visualize the protection map



3 Why “retain-aware” matters

If a module is crucial for both the forgotten facts and useful knowledge, strongly restricting it might harm new-task learning. The retain gradients give us a simple way to notice that overlap before imposing a strong constraint.

What could falsify our idea?

If random module rankings perform just as well, or if protected modules are not associated with smaller recovery, then the scoring rule has not contributed useful information. A negative result is still valuable, but it would not justify the proposed method.

SECTION 11 / METHOD B

How do we restrict learning selectively?

Use a weighted penalty on changes after the unlearned checkpoint.

1 Normal later-task fine-tuning

L_new = CE(new-task examples)

Without any defense, the optimizer updates trainable LoRA modules to solve the new task.

2 Add a plasticity penalty around M2

$L_{RASP} = L_{new} + \lambda \cdot \sum_i w_i \cdot \| U_i(t) - U_i(M2) \|^2$

U_i is the effective LoRA update for module i; w_i is a normalized risk weight.

The model still learns from **L_new**. The second term makes changes more expensive in modules assigned a larger risk weight. Low-risk modules receive little or no extra penalty. Lambda controls the overall strength.

3 Why anchor the penalty at M2?

M2 is the state after unlearning that we are attempting to keep stable. Penalizing distance to M1 (the earlier fact-trained model) would push back toward the very behavior we tried to suppress. We compare later updates to the **post-unlearning** model, not the original one.

4 LoRA detail that avoids a subtle error

Protect the effective update, not an unrelated new adapter

A LoRA module has update $U = B \times A$, scaled by LoRA's alpha/rank convention. If possible, compare its **effective update matrix** at time t with the one saved at M2. Penalizing individual A and B matrices is cheaper but parameterization-dependent; document that approximation if used.

5 Potential failure

If lambda is too high, even retained or novel information may become difficult to learn. If lambda is too low, target recovery may remain unchanged. This trade-off is precisely what the experiment needs to measure.

SECTION 11 / CHOICES

Three variants to compare before adding complexity

Start with fixed module scores. Dynamic re-scoring and topology belong to later ablations.

1 A controlled progression of methods

Variant	Implementation	What it tests
A. Uniform anchoring	Apply the same penalty weight to all LoRA modules.	Does generic regularization suffice?
B. Random anchoring	Assign strong weights to randomly chosen modules.	Does location matter?
C. Forget-only	Rank with forget gradients; ignore retain gradients.	Does retain-awareness add value?
D. RASP	Use clipped forget/retain gradient ratio.	Is the proposed scoring useful?

2 What a first implementation could look like

```
# Pseudocode: after baseline unlearning (M2)
theta_M2 = snapshot_lora_updates(model)
risk = score_modules(model, forget_loader, retain_loader)
weights = normalize_and_clip(risk)

for new_batch in new_task_loader:
    loss_new = answer_only_ce(model, new_batch)
    loss_protect = weighted_lora_drift(model, theta_M2, weights)
    loss = loss_new + lambda_reg * loss_protect
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

3 What the pseudocode deliberately leaves out

- Exact tokenizer/chat templates, target-completion masks, and batching.
- Mixed precision, optimizer-state checkpointing and gradient accumulation.
- Risk-score calibration and fixed development-time hyperparameter selection.

Do not confuse this with unlearning

RASP acts **after** an existing unlearning algorithm. Its job is to alter subsequent legitimate training. If the initial unlearning failed, anchoring that model will only preserve the failure.

SECTION 12 / IMPLEMENTATION

A realistic code and experiment blueprint

Separate the stages so failures can be debugged without rerunning everything.

1 Suggested repository structure

```
rasp-project/
  configs/           # model, data, optimizer and seed
  data/              # synthetic entities and prompt splits
  train_facts.py     # M0 -> M1
  unlearn.py         # M1 -> M2
  probe_recovery.py  # ordinary later SFT recovery pilot
  score_modules.py   # gradient importance at M2
  train_guarded.py   # uniform / random / RASP
  evaluate.py        # recovery, utility, traces, VRAM
  results/          # CSV, plots and checkpoints
```

2 Implementation sequence (in dependency order)

Milestone	Deliverable before moving on
01 - Data	Reproducible fictional fact generator; entity-level splits.
02 - Knowledge injection	M1 learns held-out direct and multi-hop target prompts.
03 - Unlearning baseline	M2 forgets targets and retains sufficient general utility.
04 - Recovery pilot	At least one later SFT condition shows nontrivial recovery.
05 - Module scoring	Reproducible per-module F/R gradient values.
06 - Guarded SFT	Uniform, random, forget-only, and RASP branches.
07 - Evaluation	Graphs, matched-cost baselines, three-seed results if feasible.

Minimum viable project

Only milestone 04 establishes the basic challenge; milestones 05-07 test the new method. Avoid topology libraries or SAE training until these results are available.

SECTION 13 / FAIR COMPARISONS

Which baselines must our idea beat?

Comparison quality matters more than model size.

1 Main experiments

Branch	During new-task fine-tuning	Reason it exists
No protection	Ordinary LoRA fine-tuning.	Measures natural recovery.
Uniform penalty	Same strength for each module.	Controls for overall update reduction.
Random protection	Risk-like weights shuffled among modules.	Controls for mere sparsity/selectivity.
Forget-only score	Protect high forget-gradient modules.	Tests whether retain data adds value.
RASP	Protect high forget/retain ratio modules.	Tests the proposed contribution.
Optional fixed freezing	Freeze a chosen set of modules.	Checks whether overly restrictive protection harms learning.

2 Hyperparameter controls

- Use the **same M2 checkpoint**, same new-task data and same evaluation checkpoints.
- Tune lambda with a development split for **all** regularized methods; do not weaken the baselines deliberately.
- Compare at **matched recovery**, **matched new-task accuracy** and comparable compute where possible.
- Repeat main runs across several seeds when resources permit.

3 Ablations that locate a real contribution

Ablation	What it answers
Retain-aware vs forget-only	Does retain importance improve the trade-off?
Block vs LoRA-projection scoring	What granularity is necessary?
Top-k mask vs soft scores	Must a module be “protected” or can weights vary smoothly?
Unrelated vs same-structure SFT	Does the benefit generalize across recovery triggers?

SECTION 14 / MEASUREMENT

How do we know whether RASP worked?

No single accuracy or refusal metric proves durable unlearning.

1 Primary outcomes

Metric	Question it answers	Desired direction
Recovery rate	How many M2-forgotten targets reappear after later tuning?	Lower
New-task accuracy	Can the model solve unseen questions from the new task?	Higher
Retain-set accuracy	Did unrelated previously learned facts survive?	Higher
Training overhead	How much extra GPU work or time did the defense add?	Lower

2 A precise recovery denominator

Recovery@t = (# M2-forgotten targets recovered by step t) / (# M2-forgotten targets)

Report the number of targets eligible for this metric; freeze the criterion before final testing.

For example, if 20 targets meet the M2 forgetting rule and 6 are later recovered, Recovery@t = 6/20 = 30%. A model that never learned a target is not evidence of successful unlearning.

3 The four plots we want in the dissertation

- Recovery rate versus continued fine-tuning steps.
- New-task accuracy versus recovery (the key trade-off plot).
- Recovery versus added GPU time or backward passes.
- Module risk scores versus observed recovery-relevant change, to validate the selection heuristic.

4 What an honest positive result looks like

A credible claim, if supported

At matched new-task accuracy, RASP recovers fewer forgotten targets than uniform and random anchoring; or at matched recovery, it preserves higher new-task accuracy with modest overhead. Differences should be stable across seeds, not a one-run accident.

SECTION 15 / FAILURE HANDLING

What if experiments do not cooperate?

Pre-agreed decisions help avoid spending months on a weak premise.

1 Failure table

Observation	Likely cause	Responsible next step
M1 cannot recall facts	Knowledge injection insufficient	Check labels, template and tiny-subset learning.
M2 does not forget	Unlearning baseline failed	Check gradient sign, masking and alpha/beta.
M2 loses all utility	Forgetting too aggressive	Reduce pressure and improve retain term.
No recovery after new SFT	No measurable challenge	Try planned same-structure condition; report it.
All penalties hurt learning	Over-constrained plasticity	Tune lambda; reduce selected modules.
Random ≈ RASP	Risk score adds little	Revisit gradient heuristic or report negative result.
Benefit disappears across seeds	Unstable evidence	Increase replicates; avoid strong claims.

2 Two particularly important conceptual risks

Risk A - knowledge is distributed

An LLM does not necessarily store one fact in one module. Protection of selected modules might simply force learning through other pathways, or it might restrict valuable shared computation. Circuit claims require more than gradient magnitude.

Risk B - this is a cooperative training guard

The method assumes the subsequent fine-tuning pipeline applies the penalty. Someone with unrestricted weights could remove the protection. The experiment cannot establish certified removal of real private information.

3 The go / no-go rule

Proceed beyond the pilot only if M2 forgetting is measurable, later fine-tuning creates a nontrivial recovery challenge, and a carefully tuned selective protection shows a benefit over strong simple controls.

SECTION 16 / PROPOSAL AND PAPER

What would we tell the project guide?

A precise incremental contribution is better than a vague claim of neuro-inspired novelty.

1 A concise proposal paragraph

For a project synopsis

This project investigates the persistence of targeted knowledge unlearning in small reasoning-capable language models subjected to legitimate downstream fine-tuning. Inspired by selective synaptic plasticity, we propose a lightweight retain-aware scoring procedure to identify LoRA modules that may be sensitive to target recovery, followed by a weighted regularization term that limits their drift from the unlearned checkpoint. We evaluate direct and multi-hop recovery, reasoning-task learning, retained utility, and computational overhead against uniform, random and forget-only regularization baselines. The approach is presented as a testable post-unlearning safeguard, not as certified deletion or a fundamentally new unlearning algorithm.

2 What is incremental, and what is potentially useful?

Already established	Potential measured contribution
Parameter anchoring and importance weighting	A cheap recovery-sensitive, retain-aware module selection study.
LLM forgetting and relearning research	Evaluation focused on post-unlearning reasoning SFT.
Localized unlearning and protection	Evidence that selective protection helps at matched utility.

3 What to include in the dissertation

- Introduction, problem setup, threat model and definition of observed forgetting.
- Review of unlearning, benign relearning, EWC/SSD and LLM localization studies.
- M0/M1/M2 pipeline, dataset splits, proposed score and loss.
- Baseline comparisons, multi-hop evaluations, ablations and uncertainty.
- Negative findings, computational limitations and ethical scope.

The closing research principle

A successful major project does not have to invent an entirely new field. It should establish a clear problem, a reproducible method, strong controls and claims no broader than its evidence.

APPENDIX / QUICK REFERENCE

Glossary you will actually use

Keep this page beside you when reading papers or building the code.

Term	Meaning in this project
Machine unlearning	Reducing targeted training influence or target behavior without full retraining.
Benign relearning	Recovery of suppressed information during later legitimate fine-tuning.
Selective plasticity	Giving different components different degrees of freedom to update.
Retain-aware	Considering useful knowledge when deciding where to restrict updates.
LoRA module	A small trainable low-rank update associated with a model projection.
M0 / M1 / M2	Base / synthetic-fact-trained / post-unlearning model states.
F / R / N / T	Forget / retain / new-task / held-out test datasets.
Gradient magnitude	How sensitive a training loss is to the chosen parameters locally.
Module risk score	Proposed heuristic ratio of forget to retain gradient sensitivity.
Anchoring / regularization	Penalty for moving a selected update away from its M2 value.
Lambda	Scalar controlling how strongly updates are restricted.
Recovery rate	Fraction of targets forgotten at M2 that reappear after later training.
LoRA effective update	The scaled low-rank matrix product B × A.
Matched utility	Comparison at a similar level of useful task performance.
Ablation	Change one method element to see what caused any difference.
Threat model	Explicit assumptions about who controls the weights and training pipeline.

One-sentence project summary

Identify which parts of an unlearned LLM appear most sensitive to forgotten knowledge, then let those parts change less during later fine-tuning while keeping the rest able to learn.

APPENDIX / LINKED REFERENCES

Papers, datasets and useful documentation

Click any link to open the original source. Publication status is stated as verified in November 2026.

Hu et al. — Unlearning or Obfuscating? (ICLR 2025)
proceedings.iclr.cc/paper_files/paper/2025/hash/18fd48d9cbbf9a20e434c9d3db6973...
Evidence that benign additional training can recover forgotten outputs.

Yoon et al. — Rethinking Benign Relearning (ICLR 2026)
proceedings.iclr.cc/paper_files/paper/2026/hash/7b1152f0646c35d9dffed261978a58...
Shows the role of syntactic similarity in recovery.

Wang et al. — Reasoning Model Unlearning (EMNLP 2025)
aclanthology.org/2025.emnlp-main.220/
Reasoning-trace leakage and reasoning-aware unlearning.

Yoon et al. — R-TOFU (EMNLP 2025)
aclanthology.org/2025.emnlp-main.265/
Benchmark for direct answers and intermediate reasoning traces.

Wang et al. — Invariance Makes LLM Unlearning Resilient (ICML 2025)
proceedings.mlr.press/v267/wang25en.html
A strong existing approach to post-unlearning fine-tuning resilience.

Kirkpatrick et al. — Elastic Weight Consolidation (PNAS 2017)
pmc.ncbi.nlm.nih.gov/articles/PMC5380101/
Classical synaptic-consolidation-inspired continual-learning regularization.

Foster et al. — Selective Synaptic Dampening (AAAI 2024)
ojs.aaai.org/index.php/AAAI/article/view/29092
Existing forget/retain parameter-importance method.

Chen et al. — CLUE (ICLR 2026)
zodiarkhangchen.com/publications/iclr2026/
Circuit-level localization of forget and retain components.

Tian and Zhou — ALMPU (AAAI 2026)
ojs.aaai.org/index.php/AAAI/article/view/39017
Attention-head localization and relearning-robust unlearning.

Li et al. — Update-Matched Unlearning (October 2026 preprint)
arxiv.org/abs/2610.21230
Plasticity versus resistance to recovery; still a preprint here.

APPENDIX / IMPLEMENTATION RESOURCES

Open resources and final research checklist

Use these resources after understanding the method; avoid blindly copying large pipelines.

Qwen3-0.6B - model card
huggingface.co/Qwen/Qwen3-0.6B
Open-weight small model; verify thinking-mode usage.

TOFU - fictitious unlearning benchmark
arxiv.org/abs/2401.06121
External synthetic-fact unlearning benchmark.

PEFT / LoRA - Hugging Face documentation
huggingface.co/docs/peft/main/conceptual_guides/lora
Adapter mechanics and trainable modules.

Transformers PEFT integration
huggingface.co/docs/transformers/peft
Practical adapter saving and resuming.

OpenUnlearning - reference code
github.com/locuslab/open-unlearning
Known unlearning baselines; check model compatibility.

GSM8K - reasoning tasks
huggingface.co/datasets/openai/gsm8k
Secondary later-task evaluation if model capacity allows.

Checklist for moving from idea to publication

- Baseline recovery demonstrated without RASP.
- M2 starts identical for every branch; data leakage checks pass.
- Risk scores reproducible and compared with randomized scores.
- New-task utility preserved at a comparable recovery target.
- Compute, seed variance, limitations and nearest prior art reported.

Status and boundaries

This guide is an **implementation proposal**, not a completed experiment. No numerical performance improvement is asserted. Neuroplasticity provides inspiration; topological analysis is optional rather than required. Last literature check: November 2026.